Combining a Dilated CNN with Classical Algorithms for Music Genre Classification

Pranav Prabhakar (PES1UG24AM463), Haneesh E (PES1UG24AM471) | UE24CS352A Machine Learning, PES University

1. Problem statement

Automatic music genre classification is needed to organise large audio collections and power recommendation and playlist systems. Genres are loosely defined and audio is high-dimensional (one 30 s clip is ~660k samples), which makes this hard for classical models trained on hand-crafted inputs. Following Li, Xue & Zhang (CS229, 2018), we ask: *can a small Dilated CNN, used as a learned feature extractor, make classical classifiers (Softmax Logistic Regression, GDA, Random Forest, SVM) more accurate, possibly more accurate than the CNN itself?*

2. Dataset

We use **GTZAN** (Tzanetakis & Cook, 2002): 1000 30-second, 22.05 kHz mono .wav clips in 10 genres. As in the paper we keep 5 genres (**classical, hip-hop, metal, pop, blues**), giving 500 clips (100 per genre). We use one stratified 80/20 split (400 train / 100 test, seed 42) for every experiment. Each clip becomes a **20 × 1290 MFCC matrix** (librosa, n_fft 2048, hop 512, cropped/tiled to 1290 frames), i.e. 25,800 values when flattened.

3. Approach

- **Baselines:** the flattened MFCC vector (25,800-d), and its top-50 PCA components, fed to LR (multinomial), GDA (shared covariance = LDA), RF (300 Gini trees, max depth 7) and an RBF-SVM (one-vs-one, $\gamma = 1/(d \cdot \mathrm{Var}\ X)$). All inputs are standardised with training statistics.
- **Dilated CNN:** the paper's final 2-unit 1D architecture over time, with the 20 MFCCs as channels. Unit 1: Conv1d(8 filters, k = 16, dilation 8) → ReLU → AvgPool 32 → Dropout 0.5 → BatchNorm. Unit 2: Conv1d(24, k = 16, dilation 2) → ReLU → AvgPool 4 → Dropout → BatchNorm. Then a dense softmax layer. Dilation lets each unit-1 filter span 121 frames (≈2.8 s) with only 16 taps; the whole network has 5,973 parameters. Trained with Adam (lr 10^{-3}), cross-entropy, batch 32, for a fixed 150 epochs. The test set is never used for model selection.
- **Combination:** the trained DCNN's unit-1 (8 × 40 = **320-d**) and unit-2 (24 × 2 = **48-d**) activations become the inputs to the four classical models, giving 4 feature sets × 4 algorithms. Because one 100-clip test set is noisy (±3 points), we also repeat DCNN training plus the classical models over 5 seeds.

4. Implementation

Python 3.13 with **librosa** (features), **PyTorch** (DCNN), **scikit-learn** (classical pipelines, PCA), matplotlib/seaborn (figures) and a **Gradio** web app for the live demo (pick a held-out clip or upload/record audio and see the DCNN probabilities next to all 8 classical predictions). The code is modular (`src/` library, `scripts/` for each stage, `run_all.py` end to end) and the full pipeline runs on a laptop CPU in about 5 minutes. Repository: github.com/pranav-prabhakar-dev/music-genre-dcnn-classical.

5. Results

Input features (dim)	split	LR	GDA	RF	SVM	5-seed test mean ± std (LR / GDA / RF / SVM)
Raw MFCC (25800)	train	1.00	0.89	1.00	0.99	– (does not depend on the DCNN seed)
	test	0.84	0.83	0.74	0.79	
PCA of raw (50)	train	1.00	0.86	1.00	0.93	– (does not depend on the DCNN seed)
	test	0.85	0.79	0.82	0.83	
DCNN layer 1 (320)	train	1.00	1.00	1.00	0.97	0.86±0.02 / 0.60±0.08 / 0.86±0.02 / 0.89±0.02
	test	0.88	0.61	0.87	0.88	
DCNN layer 2 (48)	train	1.00	0.99	1.00	0.99	0.87±0.03 / 0.85±0.02 / 0.85±0.05 / 0.87±0.02
	test	**0.89**	0.86	0.87	0.86	
DCNN alone (5,973 params)	train / test	0.99 / **0.89**				0.87 ± 0.03

Table 1: Train/test accuracy of each classical algorithm on each input (seed 42), plus the 5-seed mean ± std test accuracy for DCNN features. Best single-run test accuracy in bold.

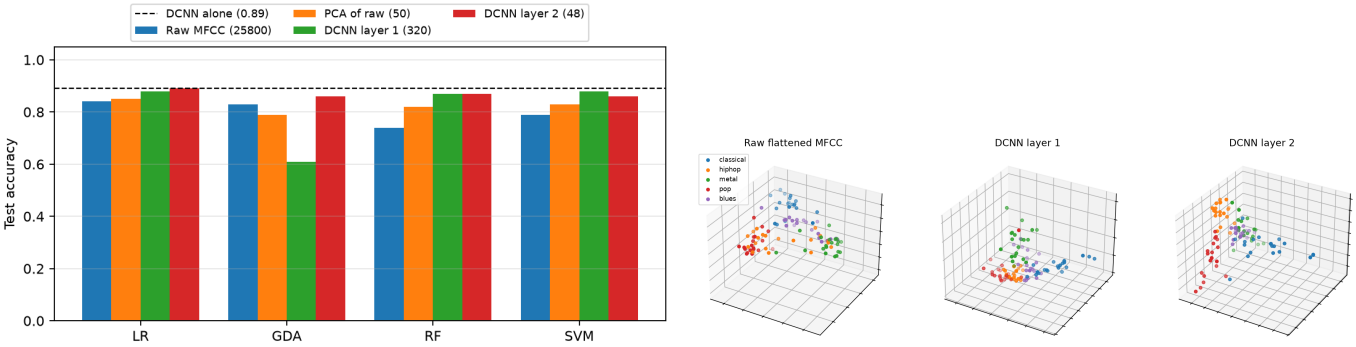
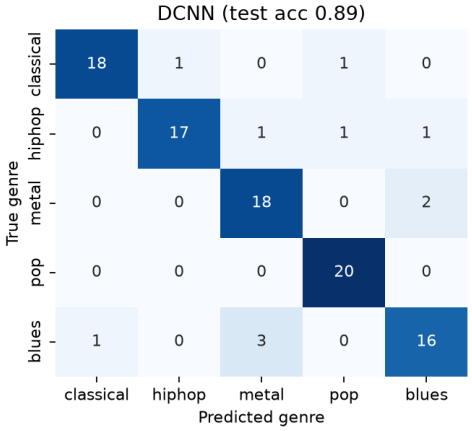


Figure 1: Left, test accuracy by algorithm and input. Right, 3D PCA of the test clips in each feature space. Genres separate more cleanly in the DCNN layers than in the raw MFCCs.

- **DCNN features help almost everywhere.** RF improves from 0.74 to 0.87, SVM from 0.79 to 0.88, and LR from 0.84 to 0.89. The 48-d layer-2 code beats the 50-d PCA code for every algorithm, so the CNN learns a much more discriminative low-dimensional representation than linear variance-based compression.
- **Classical model beats the CNN.** Over 5 seeds the RBF-SVM on layer-1 features reaches 0.894 ± 0.024, above the DCNN's own softmax (0.872 ± 0.028). This agrees with the paper: a max-margin classifier on the richer, earlier representation acts as a regulariser on top of a small, imperfect network.
- **GDA fails on layer 1** (0.61, the same as the paper's 0.61). One shared Gaussian covariance over 320 correlated, pooled ReLU activations estimated from 400 clips is a poor model. On the compact layer-2 code GDA recovers to 0.86.
- **We explain the paper's SVM anomaly.** The paper reports SVM test accuracy of 0.28 / 0.21 on raw / PCA input. We reproduce this pattern (train 1.00, test 0.39 / 0.22) using *unscaled* MFCCs with $\gamma = 1/d$, scikit-learn's pre-2019 default. The kernel becomes too narrow and the SVM memorises the training set. With standardised features the same SVM scores 0.79 / 0.83.
- The DCNN alone reaches 0.89 test accuracy (paper: 0.84). Most errors are between metal, blues and hip-hop. Classical music is recognised almost perfectly (Fig. 2). In the architecture study (Table 2) a single first-layer filter is far too few (0.64; paper 0.49). Every other configuration lands at 0.85–0.91, within about two standard deviations. Unlike the paper, larger filters did not help and a third unit did not overfit, so on this data the architecture choice matters less than the paper suggests.



DCNN layers: filters, kernel, dilation, pool (d = dropout)	Train	Test (3 seeds)
1,8,4,32 | 16,4,2,8	0.91	0.64 ± 0.04
8,8,4,32 | 16,4,2,8	1.00	0.91 ± 0.01
8,8,4,d16 | 16,8,2,d4	0.99	0.88 ± 0.02
8,16,8,d32 | 24,16,2,d4 (final)	0.99	0.87 ± 0.03
8,64,8,d32 | 24,16,2,d4	0.99	0.85 ± 0.01
16,64,8,d32 | 64,16,2,d4	1.00	0.86 ± 0.01
3-layer: 8,16,8,d32 | 24,8,2,d2 | 24,4,2,d2	0.98	0.87 ± 0.01

Figure 2 (left): DCNN confusion matrix on the 100 test clips. Table 2 (right): DCNN architecture study (paper Tables V.1/V.2), mean ± std test accuracy over 3 seeds.

6. Conclusions

A small dilated CNN trained on only 400 clips is an effective feature extractor. Its activations raise the accuracy of all four classical algorithms (except GDA on the wide layer), and an SVM on those activations matches or beats the network itself, as the original paper claims. Feature preprocessing and kernel hyper-parameters matter as much as the model: the paper's dramatic SVM failure on raw features was an artefact of unscaled inputs and γ. With a 100-clip test set, differences of 1–3 points are within seed noise, so we report multi-seed averages. **Future work:** recurrent (LSTM/GRU) or attention feature extractors for the sequential structure, all 10 GTZAN genres with an artist-filtered split, data augmentation by taking several 3–5 s windows per clip, and cross-validated hyper-parameter search for the classical models.